

Controllers, APIs e integracao GitHub

Como as requisicoes entram no devBoard, quais endpoints existem e como preparar o OAuth e os webhooks do GitHub.

Material para acompanhar o codigo do trabalho academico. O catalogo foi conferido nos controllers Spring Boot do workspace em 1 de outubro de 2026.

Escopo: pacote com.devboard.controller, rotas declaradas, fluxo de autenticacao e integracao com GitHub. A regra de negocio fica nos services; este guia explica a fronteira HTTP.

Mapa rapido

1. O papel do controller e a autenticacao JWT
2. APIs de autenticacao, projetos, boards, tarefas e membros
3. Rotas especificas do GitHub e o endpoint de webhook
4. Credenciais, configuracao local e roteiro de teste manual

Projeto devBoard | Backend Java + Spring Boot | Frontend Angular

1. Como ler o pacote controller

Um controller é a porta HTTP da aplicação. Ele declara a rota, recebe parâmetros e JSON, valida formatos com `@Valid`, pega a identidade autenticada com `@AuthenticationPrincipal` e delega o trabalho a um service. Por exemplo, `TaskController.move` recebe `taskId` e `MoveTaskRequest`, obtém o usuário logado e chama `TaskService.move`.

Fluxo típico da requisição

Cliente Angular -> Controller -> Service -> Repository/entidades -> PostgreSQL. Para resposta, o caminho volta como DTO. O controller não deve decidir regra de permissão nem acessar o repository diretamente.

Autenticação e erros

A maior parte de `/api/**` exige `Authorization: Bearer {JWT}`. O filtro JWT valida o token e o Spring disponibiliza `SecurityUser` ao controller. O `SecurityConfig` libera algumas rotas públicas: registro e login, OAuth GitHub, recuperação de senha, consulta pública de convite e webhook GitHub. O GET `/api/health` está liberado pela configuração, mas não há controller de health dentro do pacote auditado.

Falhas de validação e de negócio são convertidas pelo `GlobalExceptionHandler` em JSON `ApiError`. Os status comuns são 400 (entrada/regra inválida), 401 (não autenticado), 403 (sem permissão), 404 (recurso ausente), 409 (conflito), 429 (limite) e 503 (serviço externo).

Notação: `{projectId}`, `{boardId}`, `{taskId}` e IDs na rota. Query parameters aparecem depois de `?`. Corpos de request e responses usam DTOs, não entidades JPA.

2. Autenticação e recuperação de senha

Método e rota	Acesso	Finalidade, entrada e resposta
POST <code>/api/auth/register</code>	Público	Cria conta. JSON <code>RegisterRequest</code> ; responde <code>AuthResponse</code> com 201.
POST <code>/api/auth/login</code>	Público	Autentica com <code>LoginRequest</code> ; responde <code>JWT/AuthResponse</code> com 200. Limite: 10 por hora por IP.
GET <code>/api/auth/github/login?redirectUri=...</code>	Público	Inicia OAuth: cria state e redireciona para o GitHub (302).
GET <code>/api/auth/github/callback?code=...&state=...</code>	Público	Valida state, troca code por token e redireciona para o frontend (302).
POST <code>/api/auth/logout</code>	JWT	Retorna confirmação. Como o JWT é stateless, o frontend descarta o token local.
GET <code>/api/auth/me</code>	JWT	Retorna o perfil do usuário autenticado.
PUT <code>/api/auth/me</code>	JWT	Atualiza o perfil com <code>UpdateProfileRequest</code> .
POST <code>/api/auth/change-password</code>	JWT	Troca senha com <code>ChangePasswordRequest</code> ; valida entrada.
POST <code>/api/auth/forgot-password</code>	Público	Solicita email de recuperação. Resposta genérica; limite 3/h por email.
GET <code>/api/auth/reset-password/validate-token?token=...</code>	Público	Verifica se o token de redefinição ainda é válido.
POST <code>/api/auth/reset-password</code>	Público	Redefine a senha a partir de <code>ResetPasswordRequest</code> .

OAuth GitHub e login GitHub

O controller apenas faz os redirecionamentos. `GithubOAuthService` constrói a URL de autorização, usa state de uso único, troca code por access token, consulta perfil e email verificado, associa ou cria o usuário e monta o retorno do frontend. O token do GitHub é usado também nas operações de repositório autorizadas pelo usuário.

3. Projetos, boards e colunas

Método e rota	Acesso	Finalidade, entrada e resposta
POST /api/projects	JWT	Cria projeto com CreateProjectRequest; responde ProjectResponse com 201.
GET /api/projects?archived=false&page=0&size=20	JWT	Lista projetos paginados; responde PageResponse<ProjectSummaryResponse>.
GET /api/projects/{projectId}	JWT	Consulta projeto e dados de participacao.
PUT /api/projects/{projectId}	JWT	Atualiza projeto com UpdateProjectRequest.
DELETE /api/projects/{projectId}	JWT	Arquiva o projeto; responde 204. O service tambem agenda remocao de webhooks.
POST /api/projects/{projectId}/boards	JWT	Cria board; responde BoardResponse com 201.
GET /api/projects/{projectId}/boards	JWT	Lista boards do projeto.
GET /api/boards/{boardId}?assigneeId=...&label=...&priority=...&type=...&search=...	JWT	Retorna quadro e tarefas agrupadas; filtros sao opcionais.
PUT /api/boards/{boardId}	JWT	Atualiza nome, descricao ou board padrao via UpdateBoardRequest.
DELETE /api/boards/{boardId}	JWT	Remove board; responde 204. O service agenda remocao do webhook associado.
POST /api/boards/{boardId}/columns	JWT	Cria coluna com CreateColumnRequest; responde 201.
PUT /api/columns/{columnId}	JWT	Atualiza coluna com UpdateColumnRequest.
DELETE /api/columns/{columnId}?moveTasksTo=...	JWT	Exclui coluna e pode mover tarefas para outra coluna; responde 204.
PUT /api/boards/{boardId}/columns/reorder	JWT	Reordena colunas via ReorderColumnsRequest.

O papel semantico da coluna (por exemplo BACKLOG, IN_PROGRESS, IN_REVIEW, DONE) permite que as automacoes GitHub localizem a coluna certa sem depender do nome mostrado ao usuario.

4. Tarefas, comentarios e atividades

Método e rota	Acesso	Finalidade, entrada e resposta
POST /api/tasks	JWT	Cria tarefa; CreateTaskRequest; responde TaskResponse com 201. Pode incluir githubIssueId.
GET /api/tasks/{taskId}	JWT	Consulta detalhe, comentarios e historico da tarefa.
PUT /api/tasks/{taskId}	JWT	Atualiza tarefa com UpdateTaskRequest.
PUT /api/tasks/{taskId}/move	JWT	Move tarefa via MoveTaskRequest; valida coluna e limite WIP.
DELETE /api/tasks/{taskId}	JWT	Arquiva tarefa (soft delete); responde 204.
POST /api/tasks/{taskId}/comments	JWT	Cria comentario; CreateCommentRequest; responde 201.
GET /api/tasks/{taskId}/comments?page=0&size=20	JWT	Lista comentarios paginados.
GET /api/tasks/{taskId}/activities?type=...&page=0&size=20	JWT	Lista atividades paginadas, opcionalmente filtradas por tipo.
PUT /api/comments/{commentId}	JWT	Edita comentario com UpdateCommentRequest.
DELETE /api/comments/{commentId}	JWT	Remove comentario; responde 204.

5. Labels, membros e convites

Método e rota	Acesso	Finalidade, entrada e resposta
GET /api/projects/{projectId}/labels	JWT	Lista labels do projeto.
POST /api/projects/{projectId}/labels	JWT	Cria label com LabelRequest; responde 201.
PUT /api/labels/{id}	JWT	Atualiza label via LabelRequest.
DELETE /api/labels/{id}	JWT	Remove label; responde 204.
POST /api/tasks/{id}/labels	JWT	Aplica lista JSON de IDs de labels a tarefa.
DELETE /api/tasks/{id}/labels/{labelId}	JWT	Retira label da tarefa; responde 204.
POST /api/projects/{projectId}/members/invite	JWT	Convida por email com CreateEmailInviteRequest; responde 201.
POST /api/projects/{projectId}/members/invite-link	JWT	Cria convite por link via CreateInviteLinkRequest; responde 201.
GET /api/projects/{projectId}/members	JWT	Lista membros e papeis do projeto.
GET /api/projects/{projectId}/invites?status=...	JWT	Lista convites, com filtro de status opcional.
PUT /api/projects/{projectId}/members/{memberId}	JWT	Atualiza papel via UpdateMemberRoleRequest.
DELETE /api/projects/{projectId}/members/{memberId}	JWT	Remove um membro; responde 204.
DELETE /api/projects/{projectId}/members/me	JWT	Usuario autenticado sai do projeto; responde 204.
POST /api/boards/{boardId}/members/import-github	JWT	Importa colaboradores do repositório com ImportGithubMembersRequest; executa como Callable.
POST /api/invites/accept	JWT	Aceita convite com AcceptInviteRequest.
GET /api/invites/{token}	Público	Mostra informações públicas do convite antes de aceitar.
DELETE /api/invites/{inviteId}	JWT	Revoga convite; responde 204.

A autorização real ocorre nos services e PermissionService. Em geral, consultar pede VIEWER; editar tarefa pede DEVELOPER; configurações do board e gestão de membros pedem ADMIN ou dono do projeto. JWT identifica o usuário, mas o papel no projeto decide o que ele pode fazer.

6. APIs de integracao com GitHub

Método e rota	Acesso	Finalidade, entrada e resposta
GET /api/github-repos?search=...	JWT	Lista repositorios onde a conta conectada tem acesso de escrita; filtro de texto opcional.
POST /api/boards/{boardId}/link-github	JWT	Vincula um unico repositorio ao board via LinkGithubRequest; agenda webhook/importacao.
DELETE /api/boards/{boardId}/link-github	JWT	Desvincula repositorio, limpa vinculos das tarefas e agenda remocao do webhook; 204.
POST /api/boards/{boardId}/sync-github	JWT	Agenda sincronizacao manual; responde 202; limite 1 chamada por 5 minutos por board.
GET /api/boards/{boardId}/github-settings	JWT	Le repositorio conectado e configuracoes das automacoes.
PUT /api/boards/{boardId}/github-settings	JWT	Atualiza automacoes e padrao de branch via GithubSettingsRequest.
POST /api/tasks/{id}/branch	JWT	Agenda criacao de branch no repositorio vinculado; CreateBranchRequest; responde 202.
POST /api/tasks/{id}/github-issue	JWT	Vincula issue existente pelo ID GitHub via LinkIssueRequest; responde 204.
DELETE /api/tasks/{id}/github-issue	JWT	Desvincula issue da tarefa; responde 204.
POST /api/tasks/{id}/github-issue/create	JWT	Agenda criacao de issue a partir da tarefa; responde 202.

Operacoes que chamam o GitHub podem responder 202: isso confirma que o trabalho foi aceito/colocado em processamento, nao que a chamada externa ja terminou. O token usado depende do usuario que conectou/vinculou a conta ao board.

7. Webhook GitHub: entrada de eventos

Método e rota	Acesso	Finalidade, entrada e resposta
POST /webhook/github	Público + assinatura	Recebe bytes originais do payload. Espera X-Hub-Signature-256, X-GitHub-Delivery e X-GitHub-Event. Valida HMAC-SHA256, identifica repositório/board, grava entrega idempotente e agenda processamento. Responde 202.

O endpoint não recebe JWT. A autenticidade vem da assinatura calculada sobre o corpo exato da requisição. Com assinatura inválida, responde 401. Para repositório desconhecido, projeto arquivado ou evento não suportado, descarta o payload e devolve 202 ao GitHub. Existe limite de 100 requisições por minuto por IP.

Eventos assinados

push: associa commits a tarefa pela branch ou referência #`{id}`, registra commit e pode mover a tarefa. pull_request: relaciona o PR a tarefa e reage a abertura, atualização, merge, fechamento e reabertura. issues: importa issue, sincroniza estado, labels e responsável conforme o board.

Caminho de uma entrega

1. O GitHub envia POST /webhook/github com JSON e cabeçalhos.
2. GithubWebhookService valida X-Hub-Signature-256 usando GITHUB_WEBHOOK_SECRET.
3. O repository.id do payload localiza o board vinculado.
4. X-GitHub-Delivery é salvo como chave única para impedir duplicidade.
5. Um evento assíncrono processa o payload depois do commit da requisição.
6. GithubEventProcessor procura tarefas somente dentro do board daquele repositório.
7. Mudanças registram atividade e, quando apropriado, agendam operação inversa no GitHub.

Vínculo automático do hook

Ao vincular repositório ao board, BoardGithubService salva o vínculo local e publica um job. GithubJobService cria no repositório um webhook com content type JSON, segredo compartilhado e eventos push, pull_request e issues. Portanto, para o fluxo implementado, não é necessário cadastrar manualmente o webhook no repositório; o servidor precisa estar acessível pelo GitHub na URL configurada.

Separação de credenciais

GITHUB_CLIENT_ID e GITHUB_CLIENT_SECRET identificam a aplicação OAuth. O token GitHub de cada usuário é obtido no fluxo OAuth e guardado cifrado usando ENCRYPTION_KEY. GITHUB_WEBHOOK_SECRET é um segredo que você mesmo gera para assinar os webhooks; não é emitido pelo GitHub nem é igual ao client secret.

8. Preparar credenciais no GitHub - passo a passo

A. Criar a OAuth App

1. Entre no GitHub com uma conta que possa administrar as configurações da conta ou organização.
2. Abra Settings -> Developer settings -> OAuth Apps -> New OAuth App.
3. Informe nome do app, URL inicial do devBoard e callback URL.
4. Para desenvolvimento local, o callback esperado no projeto é `http://localhost:8080/api/auth/github/callback`. Para implantação, use a URL pública HTTPS equivalente e configure GITHUB_OAUTH_CALLBACK_URL com o mesmo valor.
5. Registre o app, copie Client ID e use Generate a new client secret para gerar o segredo. Guarde-o imediatamente em local seguro.

B. Configurar permissões e credenciais

O código solicita os escopos `repo` e `user:email`. `repo` autoriza operações nos repositórios aos quais o usuário der acesso; `user:email` permite localizar o e-mail verificado. O usuário precisa ter permissão de escrita no repositório escolhido para criar hook, branch e issue. OAuth gera um token por usuário depois do consentimento; não crie um PAT para substituir esse fluxo.

Continuacao: configuracao local e validacao

C. Definir o segredo do webhook

Gere uma string aleatoria de alta entropia (por exemplo, 32 bytes aleatorios em hexadecimal) e coloque o mesmo valor no ambiente do backend em `GITHUB_WEBHOOK_SECRET`. O sistema usa esse valor para criar o hook e validar a assinatura recebida. Nao cole o segredo no codigo, no frontend ou em commit.

D. Tornar o endpoint acessivel

O padrao local `http://localhost:8080/webhook/github` funciona apenas entre processos da sua maquina; o GitHub na internet nao consegue alcanca-lo. Para teste real, publique o backend com HTTPS ou use um tunel HTTPS de desenvolvimento. Configure `GITHUB_WEBHOOK_URL` com a URL publica terminando em `/webhook/github`. Em producao, use dominio e certificado TLS estaveis.

Variaveis de ambiente do fluxo GitHub

Variavel	Uso	O que configurar
<code>GITHUB_CLIENT_ID</code>	OAuth App	Client ID exibido na pagina da OAuth App.
<code>GITHUB_CLIENT_SECRET</code>	OAuth App	Client secret gerado para a aplicacao; fica apenas no backend.
<code>GITHUB_OAUTH_CALLBACK_URL</code>	OAuth App	Callback cadastrado no GitHub. Local: <code>http://localhost:8080/api/auth/github/callback</code> .
<code>APP_BASE_URL</code>	Frontend	Origem do Angular, usada para redirecionar apos OAuth; local tipico <code>http://localhost:4200</code> .
<code>GITHUB_WEBHOOK_SECRET</code>	Webhook	Segredo aleatorio criado por voce; armazenado no backend e enviado ao GitHub ao criar o hook.
<code>GITHUB_WEBHOOK_URL</code>	Webhook	URL publica acessivel do backend; termina em <code>/webhook/github</code> . Em dev precisa de tunel HTTPS.
<code>ENCRYPTION_KEY</code>	Criptografia	Chave AES-GCM de 32 bytes, informada em Base64 (<code>openssl rand -base64 32</code>). Mantenha estavel e protegida; perder/trocar a chave exige reconectar contas.

O backend tambem precisa de PostgreSQL e JWT configurados para funcionar: `DB_URL`, `DB_USER`, `DB_PASSWORD` e `JWT_SECRET`. Esses valores nao sao credenciais do GitHub, mas sao pre-requisitos do devBoard.

E. Fazer uma verificacao manual

1. Inicie PostgreSQL e o backend com as variaveis definidas; confirme que Liquibase aplica migrations.
2. Inicie o Angular com `APP_BASE_URL/origin` coerente.
3. Use Entrar com GitHub. Autorize a OAuth App e confirme o retorno para o frontend.
4. Abra configuracoes de um board, carregue repositorios e vincule um repositorio onde sua conta tem escrita.
5. Confira na pagina do repositorio em Settings -> Webhooks que o hook foi criado para a URL configurada e que os eventos estao habilitados.
6. Crie uma tarefa, gere branch pelo devBoard e faca push. Verifique atividade/coluna do board.
7. Em Recent Deliveries do webhook, examine status e resposta. Uma entrega aceita deve receber HTTP 202; falha de assinatura aparece como 401.

8. Estude o codigo acompanhando uma requisicao

Para aprender sem tentar ler todo o repositorio de uma vez, siga estes pares: BoardGithubController -> BoardGithubService -> GithubJobService -> GithubRepositoryClient; GithubWebhookController -> GithubWebhookService -> GithubEventListener -> GithubEventProcessor; TaskGithubController -> TaskGithubService; TaskController -> TaskService. A classe de controller mostra o contrato HTTP; o service mostra a regra e as permissoes; o client mostra como a API externa e chamada.

9. Checklist e resumo para memorizar

- Callback OAuth cadastrado no GitHub igual a GITHUB_OAUTH_CALLBACK_URL.
- Usuario autorizou a app e tem escrita no repositorio escolhido.
- GITHUB_WEBHOOK_URL e publica, HTTPS e termina em /webhook/github.
- Client secret OAuth, webhook secret e ENCRYPTION_KEY estao configurados somente no backend.
- Hook aparece nas configuracoes do repositorio e as entregas recentes retornam HTTP 202.

Pontos para lembrar

Client secret OAuth: secreto da aplicacao, usado no backend para trocar authorization code por token.

Token GitHub do usuario: credencial resultante do consentimento individual; codigo cifra antes de salvar.

Webhook secret: segredo compartilhado que autentica payloads; nao e o OAuth client secret.

202 Accepted: confirma recebimento/agendamento, nao conclusao imediata da operacao assincrona.

Referencias oficiais do GitHub

Criar OAuth App: [docs.github.com - Creating an OAuth app](https://docs.github.com/en/apps/creating-github-apps)

Validar assinatura do webhook: [docs.github.com - Validating webhook deliveries](https://docs.github.com/en/apps/webhooks/securing-webhooks)

Boas praticas para credenciais OAuth: [docs.github.com - Best practices for creating an OAuth app](https://docs.github.com/en/apps/creating-github-apps)

Fim do guia. Consulte os DTOs e services correspondentes quando precisar conferir campos e regras detalhadas.